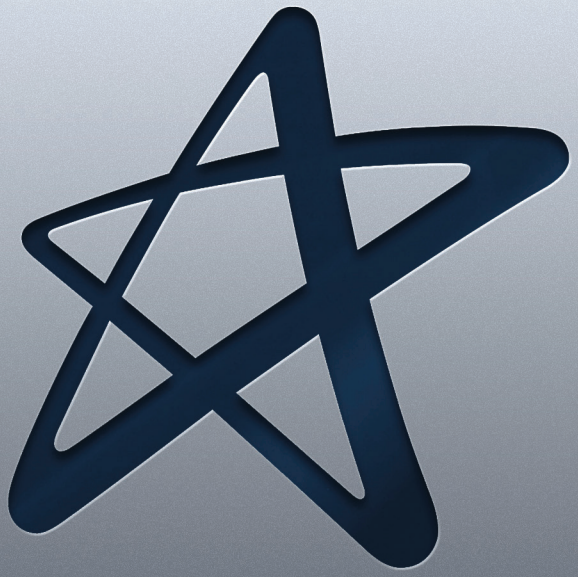
Linguagens Formais e Autômatos



Cruzeiro do Sul Virtual
Educação a distância

Material Teórico

Linguagens Regulares – Expressão Regular

Responsável pelo Conteúdo:
Prof. Dr. Cléber Silva Ferreira da Luz

Revisão Textual:
Prof.ª Dr.ª Selma Aparecida Cesarin

UNIDADE Linguagens Regulares – Expressão Regular

- Introdução;
- Expressão Regular;
- Gramática Regular;
- Propriedades das Linguagens Regulares.



OBJETIVO DE APRENDIZADO

- Apresentar os conceitos básicos sobre Linguagens Regulares.

Orientações de estudo

Para que o conteúdo desta Disciplina seja bem aproveitado e haja maior aplicabilidade na sua formação acadêmica e atuação profissional, siga algumas recomendações básicas:



Assim:

- ✓ Organize seus estudos de maneira que passem a fazer parte da sua rotina. Por exemplo, você poderá determinar um dia e horário fixos como seu "momento do estudo";
- ✓ Procure se alimentar e se hidratar quando for estudar; lembre-se de que uma alimentação saudável pode proporcionar melhor aproveitamento do estudo;
- ✓ No material de cada Unidade, há leituras indicadas e, entre elas, artigos científicos, livros, vídeos e *sites* para aprofundar os conhecimentos adquiridos ao longo da Unidade. Além disso, você também encontrará sugestões de conteúdo extra no item **Material Complementar**, que ampliarão sua interpretação e auxiliarão no pleno entendimento dos temas abordados;
- ✓ Após o contato com o conteúdo proposto, participe dos debates mediados em fóruns de discussão, pois irão auxiliar a verificar o quanto você absorveu de conhecimento, além de propiciar o contato com seus colegas e tutores, o que se apresenta como rico espaço de troca de ideias e de aprendizagem.

Introdução

Linguagem formal são mecanismos formais para **representação e especificação de Linguagens**.

As representações são realizadas por **reconhecedores** e **geradores**. Já estudamos mecanismos de geração de Linguagem e a Gramática e também.

Nesta Unidade, iremos focar nas Expressões Regulares, que são poderosos mecanismos de reconhecedores, assim como os Autômatos, já estudados na Unidade anterior.

As Expressões Regulares são utilizadas, principalmente, como descritores de Linguagens, ou seja, a partir dessas expressões, podemos identificar uma Linguagem Regular e dada uma Linguagem, podemos escrevê-la de forma simplificada, usando expressões.

A próxima seção apresenta com detalhes o conceito de Expressão Regular.

Expressão Regular

Na Teoria das Linguagens Formais, toda Linguagem Regular pode ser descrita por uma expressão chamada de **Expressão Regular**.

Expressão Regular é um formalismo denotacional que também pode ser considerado gerador, pois se pode inferir como construir ("gerar") as palavras de uma Linguagem.

Expressão Regular é definida a partir de conjuntos (Linguagens) básicos e operações de concatenação e de união. As Expressões Regulares são indicadas para a comunicação humano x humano e, também, na comunicação humano x máquina (MENEZES, 2011).

Habitualmente, uma Expressão Regular é abreviada por ER. Uma Expressão Regular sobre um alfabeto Σ é indutivamente definida como segue:

- Base de Indução.
  - » A expressão:

$$\varnothing$$

é expressão Regular e denota a Linguagem vazia:

$$\varnothing$$

  - » A expressão:

$$\varepsilon$$

é uma Regular e denota a Linguagem que contém exclusivamente a palavra vazia:

$$\{\varepsilon\}$$

» Para qualquer símbolo $x \in \Sigma$, a expressão:

$$X$$

é uma expressão Regular e denota a Linguagem que contém exclusivamente a palavra constituída pelo símbolo x:

$$\{ X \}$$

- Passo de Indução. Se r e s são Expressões Regulares e denotam as Linguagens R e S respectivamente, então:

» União.

A expressão:

$$(r+s)$$

é expressão Regular e denota a Linguagem:

$$R \cup S$$

» Concatenação.

A expressão:

$$(rs)$$

é expressão Regular e denota a Linguagem:

$$R\,S = \{uv \mid u \in R \text{ e } v \in S\}$$

- Concatenação Sucessiva.

A expressão:

$$(r^*)$$

é expressão Regular e denota a Linguagem:

$$R^*$$

Se r é uma expressão Regular, a Linguagem correspondente é dita a **Linguagem Gerada** por r, sendo denotada por:

$$L(r) \text{ ou } GERA(R)$$

É possível omitir os parênteses em uma ER. Todavia, é necessário respeitar as seguintes convenções:

- A concatenação sucessiva tem precedência sobre a concatenação e a união;
- A concatenação tem precedência sobre a união.

Exemplo

Expressão Regular

A seguir, é apresentada uma Tabela com Expressões Regulares e as correspondentes Linguagens geradas.

Tabela 1

Expressão Regular	Linguagem Gerada
Aa	Somente a palavra aa.
ba*	Todas as palavras que iniciam por b, seguido por zero ou mais a.
(a+b)*	Todas as palavras sobre {a, b}.
(a+b)*aa(a+b)*	Todas as palavras contendo aa como subpalavra.
a*ba*ba*	Todas as palavras contendo exatamente dois b.
(a+b)*(aa+bb)	Todas as palavras que terminam com aa e bb.
(a+ε)(b+ba)*	Todas as palavras que não possuem dois a consecutivos.

Vamos analisar a Linguagem gerada pela expressão (a+b)*(aa++bb):

- a e b denotam {a} e {b}, respectivamente;
- a + b denota {a} ∪ {b} = {a,b};
- (a+b)* denota {a,b}*;
- aa e bb denotam {a}{a}= {aa} e {b}{b} = {bb}, respectivamente;
- (aa+bb) denota {aa} ∪ {bb} = {aa,bb};
- (a+b)*(aa+bb) denota {a,b}*{aa,bb};

Dessa forma, GERA ((a+b)*(aa+bb)) corresponde à seguinte Linguagem: {aa,bb,a bb,baa,aaaa,abaa,abbb,baaa,babb,bbaa,bbbb....}

Expressão Regular → Linguagem Regular

Na Teoria das Linguagens Formais, o seguinte Teorema é válido Se r é uma expressão Regular.

Então, GERA(r) é uma Linguagem Regular (MENEZES, 2011).

Prova por indução

Uma Linguagem é Regular se, e somente se, é possível construir um Autômato Finito pode ser Determinístico, Não Determinístico e com Movimento Vazio, que reconheça tal Linguagem.

Dessa forma, é necessário que, dada uma expressão Regular r qualquer, é possível construir um Autômato Finito M tal que:

$$Aceita(M) = Gera(r)$$

Na construção do Autômato Finito com movimento vazio M correspondente apresentado a seguir, a demonstração de que Aceita(M) = Gera(M) é por indução no número de operadores (MENEZES, 2011).

- Base de Indução. Seja r uma expressão Regular com zero operadores, então, r só pode ser da forma:

$$r = \varnothing$$

$$r = \varepsilon$$

$$r = x \text{ (x pertencente a } \Sigma)$$

os Autômatos Finitos:

$M_1 = (\varnothing, \{q_0\}, \delta_1, q_0, \varnothing)$

$M_2 = (\varnothing, \{q_f\}, \delta_2, q_f, \{ q_f \})$

$M_3 = (\{x\}, \{q_0,q_f \}, \delta_3, q_0, \{ q_f \})$

Os Autômatos ilustrados na Figura 1 aceitam as Linguagens acima, respectivamente:

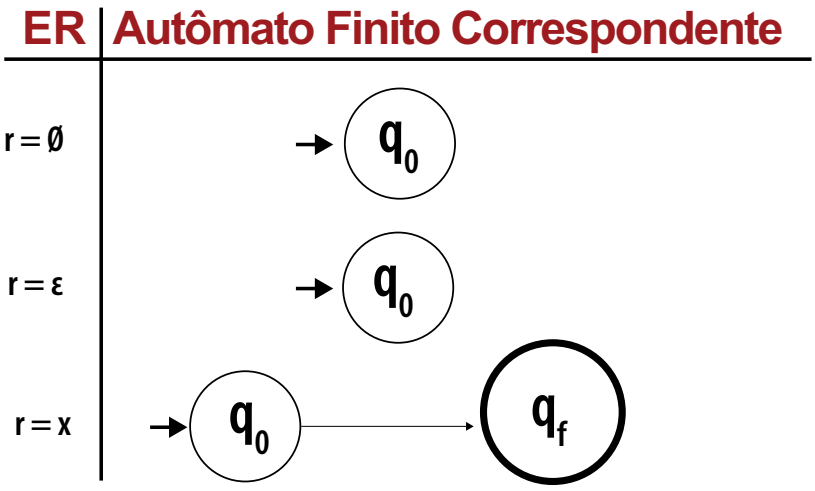


Figura 1 – Autômatos Finitos correspondentes às ER com zero operadores

- Hipótese de Indução.

  Seja r uma expressão Regular com até n > 0 operadores.

  Vamos supor que seja possível definir um Autômato Finito que aceita a Linguagem gerada por r;

- Passo de Indução.

  Seja r uma expressão Regular com n + 1 operadores. Então, r pode ser representada por um dos seguintes casos, onde r_1 e r_2 juntos possuem no máximo n operadores:

$$r = r_1 + r_1$$

$$r = r_1\ r_2$$

$$r = r_1^*$$

Assim, por hipótese de indução, é possível construir os Autômatos:

$$M_1 = (\Sigma_1, Q_1, \delta_1, q_{01}, \{q_f\}) \text{ e } M_2 = (\Sigma_2, Q_2, \delta_2, q_{02}, \{q_{f2}\})$$

que:

$$\text{Aceita}(M_1) = \text{Gera}(r_1) \text{ e Aceita}(M_2) = \text{Gera}(r_2)$$

Os Autômatos com Movimentos Vazios que aceitam a Linguagem GERA(r) para cada caso são:

$$r = r_1 + r_2.$$

O Autômato:

$$M = (\Sigma_1 \cup \Sigma_2, Q_1 \cup Q_2 \cup \{q_0, q_1\}, \delta, q_0, \{q_f\}),$$

que é apresentado na Figura 2, é tal que, a parir do estado inicial q_0, realiza transições vazias para o estado q_{01} e q_{02}.

Dessa forma, M_1 e M_2 processam de forma não determinística e, assim, é suficiente um dos módulos aceitar a entrada para o Autômato M aceitar;

$$r = r_1 r_2.$$

O Autômato:

$$M = (\Sigma_1 \cup \Sigma_2, Q_1 \cup Q_2, \delta, q_{01}, \{q_{f2}\}),$$

apresentado na Figura 2, processa os módulos M_1 e M_2 em sequência, aceita a entrada se, e somente se, o prefixo pertencer a Aceita(M_1) e o sufixo pertencer a Aceita(M_2);

$$r = r_1^*.$$

O Autômato:

$$M = (\Sigma_1 \cup \Sigma_2, Q_1 \cup Q_2, \delta, q_{01}, \{q_{f2}\}),$$

também ilustrado na Figura 2, é tal que a transição vazia de q_0 para q_f garante a aceitação da palavra vazia e o ciclo de q_f para q_{01} permite o sucessivo processamento de M_1 para assegurar o reconhecimento de duas ou mais concatenação sucessivas:

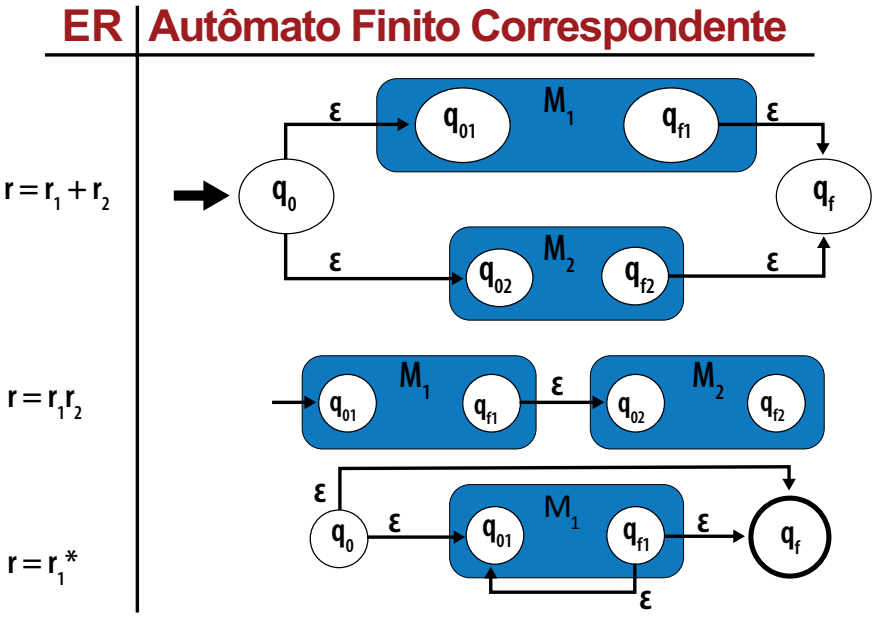


Figura 2 – Autômatos Finitos correspondentes às ER com n+1 operadores

Exemplo

Dada uma ER vamos construir o AFNε

A Figura 3 ilustra a construção dos módulos de um AFNε que aceita a Linguagem gerada pela expressão Regular a*(aa + bb).

Já a Figura 4 ilustra o autômato resultante.

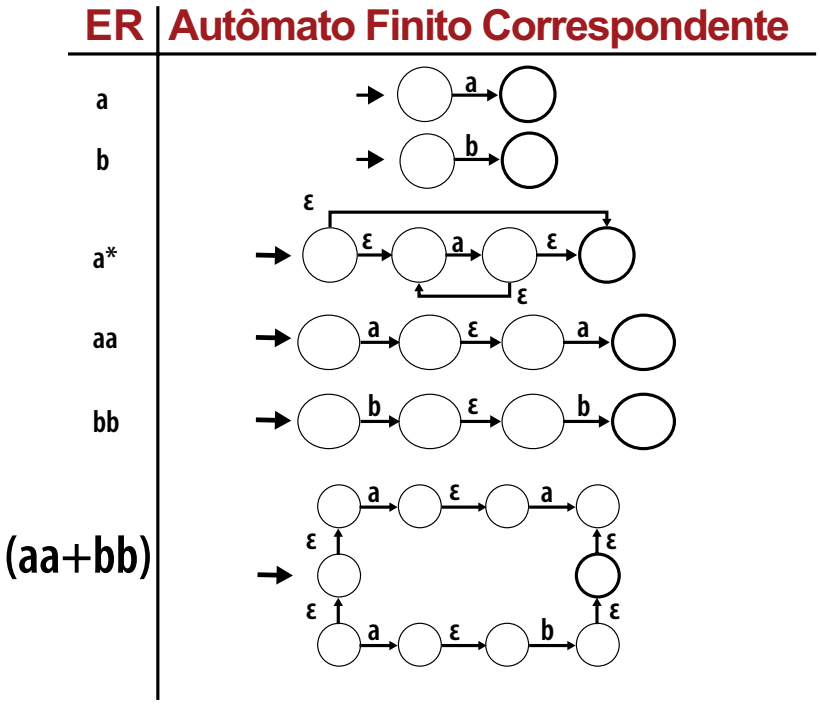


Figura 3 – ER e os correspondentes Autômatos Finitos

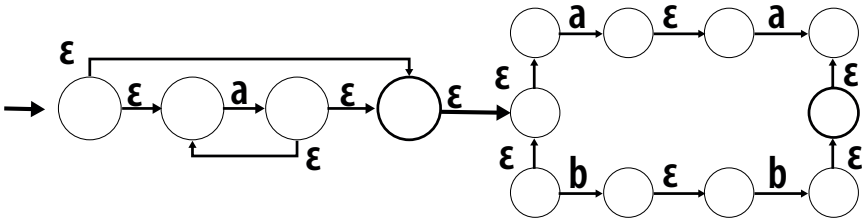


Figura 4 – Diagrama: AFNε construído a partir da ER

Gramática Regular

Uma Gramática pode ser considerada um conjunto finito de regras aplicadas sucessivamente para gerar palavras (mais detalhes na Unidade anterior).

Gramática Regular é uma Gramática em que são estabelecidas restrições específicas nas regras de produção.

Segundo Menezes (2011), há várias formas de restringir as regras de produção para definir uma Gramática Regular. Quatro delas são denominadas Gramáticas Lineares.

Vamos estudá-las a seguir.

Gramáticas lineares

Considere a Gramática G = (V, T, P, S). Sejam A e B elementos de V e w uma palavra de T*. Então, G é uma Gramática Linear se todas as suas produções se encontram em uma e em somente uma das seguintes formas:

- Gramática Linear à Direita (GLD): todas as regras de produção são da forma:

$$A \rightarrow wB \text{ ou } A \rightarrow w.$$

- Gramática Linear à Esquerda (GLE): todas as regras de produção são de forma:

$$A \rightarrow Bw \text{ ou } A \rightarrow w.$$

- Gramática Linear Unitária à Direita (GLUD): todas as regras de produção são como na Gramática Linear à Direita e, adicionalmente:

$$|w| \leq 1.$$

- Gramática Linear Unitária à Esquerda (GLUE): Todas as regras de produção são como na Gramática Linear à Esquerda e, adicionalmente:

$$|w| \leq 1.$$

É importante observar que, nas Gramáticas Lineares, o lado direito de uma produção é constituído por, no máximo, uma variável. Além dessa variável, sempre antecede ou sucede subpalavras de terminais, caso exista.

Equivalência das Gramáticas Lineares

Considerando L uma Linguagem, então, temos:

- L é gerada por uma GLD se, e somente se;
- L é gerada por uma GLE se, e somente se;
- L é gerada por uma GLUD se, e somente se;
- L é gerada por uma GLUE se.

Isto é, as diversas formas das Gramáticas Lineares são equivalentes.

Gramática Regular

Agora que já entendemos o que é uma Gramática Linear, podemos definir Gramática Regular. Uma Gramática G é denominada Regular se G é uma Gramática Linear.

Habitualmente, uma Gramática Regular é abreviada por GR.

Exemplo 1

Gramática Regular a(ab)*

A Linguagem a(ab)* é gerada pelas Gramáticas Regulares:

- Linear à Direita. G = ({S, A}, {a, b}, P, S).

  P = {S → aA

  A→ baA

  A → ε };

- Linear à Esquerda. G = ({S, A}, {a, b}, P, S)

  P = { S → Sba

  S→ a };

- Linear Unitária à Direita. G = ({S, A, B}, {a, b}, P, S)

  P = { S → aA

  A → bB

  A → ε A → aA };

- Linear Unitária à Esquerda. G = ({S, A}, {a, b}, P, S)

  P = { S → Aa S → a

  A → Sb.

Exemplo 2

Gramática Regular (a+b)*(aa+bb)

- Linear à Direita. G = ({S, A}, {a, b}, P, S)

  P = { S → aS

  S → bS

  S → A

  A → aa | bb };

- Linear à Esquerda. G = ({S, A}, {a, b}, P, S)

  P = { S → Aaa

  S → abb

  A → Aa

  A → Ab

  A → ε.

Linguagem gerada

Considere a Gramática G = (V, T, P, S). A Linguagem Gerada pela Gramática G, denotada por:

$$L(G) \text{ ou } Gera(G)$$

onde:

$$L(G) = \{w \in T^* \mid S \rightarrow w^+\}$$

Gramática Linear à Esquerda e Linear à Direita

Vamos supor que $|w| \geq 1$. Caso uma Gramática tenha simultaneamente produções do tipo $A \rightarrow wB$ e $A \rightarrow Bw$, então, a correspondente Linguagem gerada poderá não ser Regular, isto é, então, não é uma Gramática Regular.

Uma Linguagem que possui duplo balanceamento não é Regular. Formalmente, esta Linguagem é denotada como:

$$\{a^n b^n \mid n \in N\}$$

Todavia, é possível desenvolver uma Gramática com produções lineares à direita e à esquerda capaz de gerar essa Linguagem.

Gramática Regular → Linguagem Regular

O seguinte Teorema é válido, se L é uma Linguagem gerada por uma Gramática Regular, então, L é uma Linguagem Regular (MENEZES, 2011).

Prova por indução

Construindo um Autômato Finito, é possível mostrar que a Linguagem é Regular.

Vamos supor que G = (V, T, P, S), uma Gramática Linear unitária à direita.

Dessa forma, o AFNε:

$$M = (\Sigma, Q, \delta, q_0, F)$$

Onde:

$\Sigma = T$

$Q = V \cup \{q_f\}$

$F = \{ q_f \}$

$q_0 = S$.

δ é dado pela Tabela a seguir:

Tabela 2

Tipo de Produção	Transição Gerada
$A \rightarrow \varepsilon$	$\delta(A, \varepsilon) = q_f$
$A \rightarrow a$	$\delta(A, a) = q_f$
$A \rightarrow B$	$\delta(A, \varepsilon) = B$
$A \rightarrow aB$	$\delta(A, a) = B$

Vamos demonstrar que Aceita(M) = Gera(G) utilizando indução no número de derivações.

- Base de indução.

  Vamos supor que $S \rightarrow a^1$ Então, se:

  - a = ε, existe uma regra $S \rightarrow \varepsilon$, Por construção de M, $\delta(S,\varepsilon) = q_{f;}$
  - a = a, existe uma regra $S \rightarrow a$, Por construção de M, $\delta(S,a) = q_{f;}$
  - a = A, existe uma regra $S \rightarrow A$, Por construção de M, $\delta(S,\varepsilon) = A$;
  - a = aA, existe uma regra $S \rightarrow aA$, Por construção de M, $\delta(S,a) = A$;

- Hipótese de indução.

  Vamos supor que $S \rightarrow a^n$ onde, se

  » a = w, então $\delta^*(S, w) = q_{f;}$

  » a = wA, então $\delta^*(S, w) = A$;

- Passo de Indução.

  Vamos supor que $S \rightarrow a^{n+1}$. Obrigatoriamente, ocorre somente a hipótese 2.b e, portanto:

$$S \rightarrow wA^n \rightarrow a$$

Então, se:

» $a = w\varepsilon = w$, existe uma regra $A \rightarrow \varepsilon$.

Logo:

$\delta^*(S, w\varepsilon) = \delta(\delta^*(S, w), \varepsilon) = \delta(A, \varepsilon) = q_{f;}$

» a = wb, existe uma regra $A \rightarrow b$.

Logo:

$\delta^*(S, wb) = \delta(\delta^*(S, w), b) = \delta(A, b) = q_{f;}$

» a = wB, existe uma regra $A \rightarrow B$.

Logo:

$\delta^*(S, w\varepsilon) = \delta(\delta^*(S, w), \varepsilon) = \delta(A, \varepsilon) = B$;

» a = wbB, existe uma regra $A \rightarrow bB$.

Logo:

$\delta^*(S, wb) = \delta(\delta^*(S, w), b) = \delta(A, b) = B$.

Exemplo 1

Construindo um AFNε a partir de uma Gramática Regular

Vamos considerar a Gramática G = ({S, A, B}, {a, b}, P, S).

P = { $S \rightarrow aA$

$A \rightarrow bB$ |

ε $B \rightarrow aA$ }

O AFNε M que reconhece a Linguagem gerada pela Gramática G é:

$$M = (\{a, b\}, \{S, A, B, q_f\}, \delta, S, \{ q_f \})$$

A função programa δ é dada pela Tabela a seguir:

Tabela 3

Produção	Transição
$S \rightarrow aA$	$\delta(S, a) = A$
$A \rightarrow bB$	$\delta(A, b) = B$
$A \rightarrow \varepsilon$	$\delta(S, \varepsilon) = q_f$
$B \rightarrow aA$	$\delta(B, a) = A$

O Autômato M é ilustrado na Figura 5.

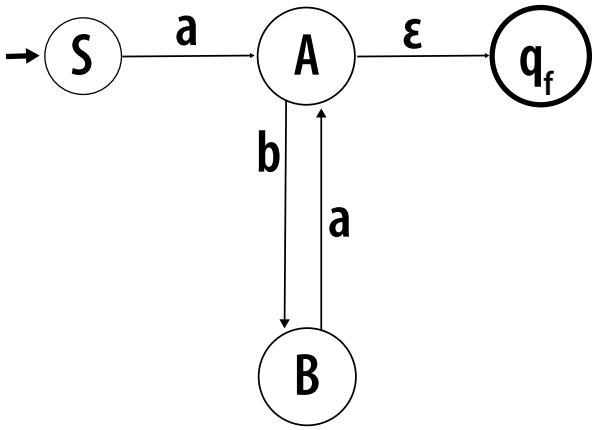


Figura 5 – Diagrama – Autômato Finito construído a partir de uma Gramática Regular

Linguagem Regular → Gramática Regular

Segundo Menezes (2011), o seguinte Teorema é válido. Caso L seja uma Linguagem Regular, então, existe uma Gramática G Regular que gera L.

Prova por indução

Se L é uma Linguagem Regular, então, existe um AFD $M = (\Sigma, Q, \delta, q_0, F)$ onde Aceita(M) = L.

O objetivo da demonstração é construir uma Gramática Linear à direita G a partir de M.

onde:

$$Gera(G) = Aceita(M),$$

isto é, onde a derivação simula a função programa estendida.

Seja a Gramática Regular:

$$G = (V, T, P, S)$$

Onde:

$V = Q \cup \{S\}$

$T = \Sigma$

P é ilustrado na Tabela a seguir, Nesta Tabela, q_i, q_k são elementos de Q e q_f é elemento de F:

Tabela 4

Transição	Produção
—	$S \rightarrow q0$
—	$qf \rightarrow \varepsilon$
$\delta(qi, a) = q_k$	$qj \rightarrow aq_k$

É utilizada indução no tamanho da palavra para demonstrar que Gera(G) = Aceita(M).

Vamos analisar a seguir:

- Base de indução.

  Seja $|w| = 0$. Vale que $S \rightarrow q_0$ é produção. Caso $\varepsilon \in$ Aceita(M), então q_0 é estado final e, por definição, vale que $q_0 \rightarrow \varepsilon$ é produção.

  Assim:

$$S \rightarrow q_0 \rightarrow \varepsilon$$

- Hipótese de Indução.

  Considere $|w| = n$ $(n \geq 1)$ e $\delta^*(q_0, w) = q$. Dessa forma, se:

  » q não é estado final, então suponha $S \rightarrow wq^n$;

  » q é estado final, então suponha $S \rightarrow wq^n \rightarrow w$.

- Passo de Indução.

  Considere $|wa| = n + 1$ e $\delta^*(q_0, wa) = p$. Então

$$\delta(\delta^*(q_0, w), a) = \delta(q, a) = p$$

  Dessa forma, obrigatoriamente, ocorre somente a hipótese 2.a e, se:

  » p não é estado final, então $S \rightarrow wq^n \rightarrow wap^1$;

  » p é estado final, então $S \rightarrow wq^n \rightarrow wap^1 \rightarrow wa^1$.

Exemplo 1

Construindo uma GR a partir de um AFD

Vamos considerar o seguinte Autômato Finito Determinístico:

$$M = (\{a, b, c\}, \{q_0, q_1, q_2\}, \delta, q_0, \{q_0, q_1, q_2\}).$$

A Gramática Regular correspondente construída é:

$$G = (\{q_0, q_1, q_2, S\}, \{a, b, c\}, P, S)$$

Onde

P é dada pela Tabela a seguir:

Tabela 5

Transição	Produção
—	$S \rightarrow q_0$
—	$q0 \rightarrow \varepsilon$
—	$q1 \rightarrow \varepsilon$
—	$q2 \rightarrow \varepsilon$
$\delta(q0, a) = q_0$	$q0 \rightarrow aq_0$
$\delta(q0, b) = q_1$	$q0 \rightarrow bq_1$
$\delta(q1, b) = q_1$	$q1 \rightarrow bq_1$
$\delta(q1, c) = q_2$	$q1 \rightarrow cq_2$
$\delta(q2, c) = q_2$	$q2 \rightarrow cq_2$

O Autômato M é ilustrado na Figura 6:

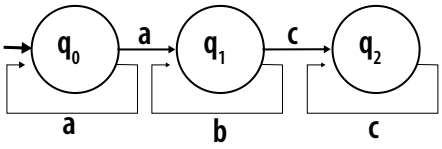


Figura 6 – AFD

Propriedades das Linguagens Regulares

As Linguagens Regulares (LR) possuem como características principais eficiência, fácil implementação e o fato de serem representadas por formalismos de pouca complexidade.

Todavia, por ser uma classe de Gramática relativamente simples, é restrita e limitada. Assim, para definir uma Linguagem Regular, é relativamente fácil.

Dessa maneira, algumas perguntas sobre Linguagens Regulares precisam ser analisadas, tais como: "Como saber se uma Linguagem é Regular ou não?", "Qual o formalismo por trás das Linguagens Regulares?".

Essa seção é destinada a responder tais perguntas (MENEZES, 2011).

A Figura 7 traduz o formalismo das Linguagens Regulares, apresentando todos os elementos da Linguagem Regular e como eles se relacionam.

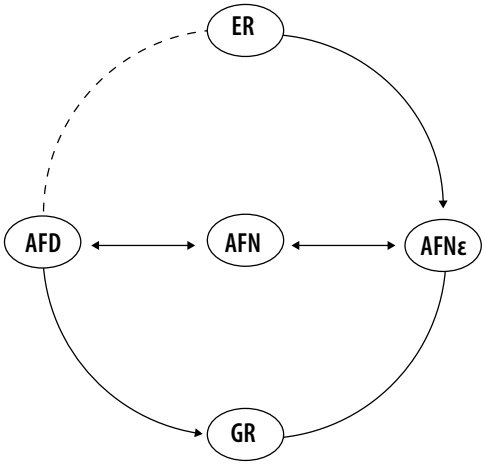


Figura 7 – Tradução dos formalismos das Linguagens Regulares

Bombeamento para as Linguagens Regulares

O Lema do Bombeamento é útil no estudo das propriedades das Linguagens Regulares.

Basicamente, a ideia é:

- Caso uma Linguagem seja Regular, então, ela é aceita por Autômato Finito Determinístico, que possui um número finito e predefinido de *n* estados;
- Caso o Autômato reconheça uma entrada w de comprimento maior ou igual ao número de estados n, obrigatoriamente o Autômato assume algum estado q mais de uma vez e, portanto, existe um ciclo na função programa que passa por q;
- Logo, a entrada w pode ser dividida em três subpalavras w = u v z tal que $|uv| \leq n$, $|v| \geq 1$, v é a parte de w reconhecida pelo ciclo;
- Tal ciclo pode ser executado ("bombeado") zero ou mais vezes. Portanto, para qualquer $i \geq 0$, u v^i z, é aceita pelo Autômato.

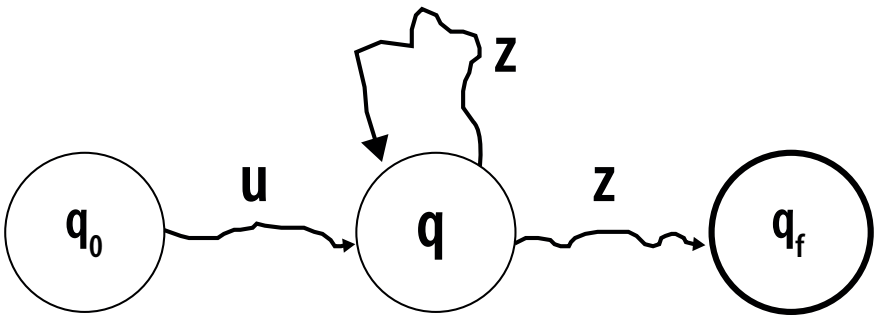


Figura 8 – Bombeamento para as Linguagens Regulares
Fonte: Adaptado de Menezes, 2011

Bombeamento para as Linguagens Regulares

Segundo Menezes (2011), o seguinte lema é válido: caso L seja uma Linguagem Regular, então:

existe uma constante n tal que,

para qualquer palavra w de L $|w| \geq n$,

w pode ser definida como w = u v z,

onde:

$|uv| \leq n$,

$|v| \geq 1$

para todo ≥ 1, u v^i z é palavra de L.

Podemos provar por prova direta

Caso L seja uma Linguagem Regular, então existe um AFD M = (Σ, Q, δ, q_0, F) onde:

Aceita(M) = L

Vamos supor que:

n seja cardinal de Q

existe w = a_1, a_2, ... a_m, uma palavra de L de comprimento m tal que m ≥ n

$\delta(q_0, a_1,) = q_1$,

$\delta(q_1, a_2,) = q_2$,

...

$\delta(q_{m-1}, a_m,) = q_m$,

Como m ≥ n, então existem r e s com $0 \leq r \leq s \leq n$,

onde:

$q_r = q_s$

$\delta(q_0, a_1, ..., a_r) = q_r$,

$\delta(q_r, a_{r+1}, ..., a_s) = q_s$,

$\delta(q_s, a_{s+1}, ..., a_m) = q_m$,

isto é, o Autômato passa mais de uma vez no estado $q_r = q_s$.

Assim:

u = a_1 .. a_r

$v = a_{r+1} .. a_s$

$z = a_{s+1} .. a_m$

como $r < s \leq n$, então:

$|v| \geq 1$ e $|uv| \leq n$

Como $q_r = q_s$, então v é reconhecida em um ciclo. Assim, para todo $i \geq 0$, u v^i z é palavra de L.

Exemplo 1

Bombeamento para as Linguagens Regulares

Para este exemplo, vamos considerar o Autômato apresentado na Figura 9. Exemplificando o Lema do Bombeamento para Linguagens Regulares, vale que;

$n = 4$

no caso particular de w = abbba, vale que:

$q_r = q_s = q_1$

u = a, v = bb, z = ba

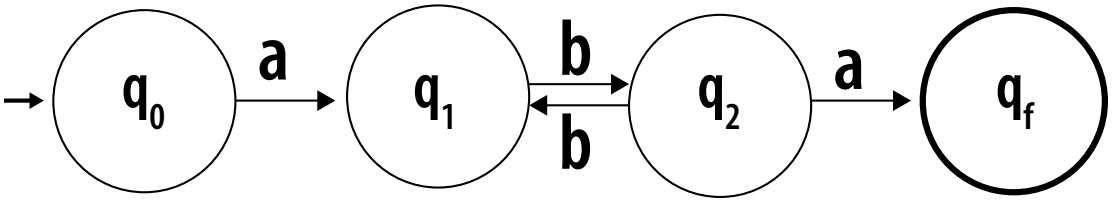


Figura 9 – AFD

Vale ressaltar que o Lema do Bombeamento garante que os formalismos Regulares são capazes de expressar diversos bombeamentos. Por exemplo, as seguintes Linguagens são Regulares:

- Duplo bombeamento: $\{a^n b^m \mid n \geq 0 \text{ e } m \geq 0\}$;
- Triplo bombeamento: $\{a^n b^m a^r \mid n \geq 0 \text{ e } m \geq 0\ r \geq 0\}$.

Definir se uma Linguagem é Regular

Para descobrir se uma Linguagem é Regular, é necessário representá-la usando um dos formalismos apresentados anteriormente, tais como, Autômato, Expressão Regular e Gramática Regular. Todavia, a prova de que é não Regular necessita ser desenvolvida para cada caso (MENEZES, 2011).

O Lema do Bombeamento é uma Ferramenta útil para Linguagens Regulares. Vamos analisar um exemplo.

Exemplo

Linguagem não Regular

A Linguagem sobre o alfabeto {a, b} "não é Regular":

L = {w | w possui o mesmo número de símbolos a e b}

Podemos provar utilizando prova por absurdo.

Suponha que L é Regular. Então, existe um AFD M com n estados que aceita L. Seja $w = a^n b^n$ palavra de L sendo $|w| = 2n \geq n$.

Pelo Lema do Bombeamento, para as Linguagens Regulares, w pode ser definida como w = u v z,

onde:

$|uv| \leq n$

$|V| \geq 1$

onde

para todos $i \geq 0$, u v^i z é palavra de L, que é um absurdo, porque, como $|uv| \leq n$, u v obrigatoriamente é composta exclusivamente por símbolos a.

Nesse caso, u v^2 z não pertence a L, por que o número de símbolos **a** será maior do que o número de símbolos b.

Operações Fechadas nas Linguagens Regulares

Operações sobre LR podem ser utilizadas para:

- Criar novas Linguagens Regulares a partir de Linguagens Regulares conhecidas (definindo assim, uma álgebra);
- Provar propriedades;
- Criar algoritmos.

A classe de Linguagens Regulares é fechada para diversas operações, tais como:

- União;
- Concatenação;
- Complemento;
- Intersecção.

Em Síntese

Autômatos Finitos e Expressões Regulares são poderosos conhecedores de Linguagens. O objetivo desta Unidade é estudar o conceito de Expressões Regulares, que são utilizadas, principalmente, como descritores de Linguagens, ou seja, a partir dessas expressões podemos identificar uma Linguagem Regular e, dada uma Linguagem, podemos escrevê-la de forma simplificada usando expressões.

As principais operações de uma ER são: união, concatenação e concatenação sucessiva.

Para toda expressão Regular existe um Autômato Finito correspondente. Dessa forma, de uma ER é possível obter um Autômato e de um Autômato é possível obter uma ER correspondente.

Uma Gramática pode ser considerada como um conjunto finito de regras aplicadas sucessivamente para gerar palavras. Gramática Regular é uma Gramática em que são estabelecidas restrições específicas nas regras de produção.

Material Complementar

Indicações para saber mais sobre os assuntos abordados nesta Unidade:

Livros

Introdução à Teoria de Autômatos, Linguagens e Computação

HOPCROFT, J. E.; ULLMAN, J. D. **Introdução à Teoria de Autômatos, Linguagens e Computação**. São Paulo: Campus, 2002;

Theory of Computation

SIPSER, M. ***Theory of Computation***. Nova Delhi: I.K. International Publishing House Pvt. Ltd., 2008;

Introdução à teoria da computação

SIPSER, M. **Introdução à teoria da computação**. Boston: Cengage, 2005;

Linguagens Formais e Autômatos: 3

MENEZES, P. B. **Linguagens Formais e Autômatos:** 3. Porto Alegre: Bookman, 2010.

Referências

GERSTING, Judith. **Fundamentos matemáticos para a ciência da computação.** 5.ed. Rio de Janeiro: LTC, 2004. 538p.

HOPCROFT, J. E.; ULLMAN, J. D.; MOTWANI, R. **Introdução à teoria de autômatos, Linguagens e computação.** Rio de Janeiro: Campus, 2001.

HOPCROFT, John E. ***Introduction to automata theory, languages and computation.*** Massachusetts: Addison-wesley Publishing Cia., 1979. 418 p.

MENEZES, P. B. **Linguagens formais e autômatos**. 6.ed. São Paulo: Bookman, 2011.

PAPADIMITRIOU, Christos H.; LEWIS, Harry R. **Elementos da Teoria da Computação**. 2.ed. Porto Alegre: Bookman, 2000. 344p.

RAMOS, Marcus Vinicios Midena; VEGA, Italo Santiago; JOSÉ NETO, João. **Linguagens formais:** teoria, modelagem e implementação. Porto Alegre: Bookman, 2009. 656p.

RAMOS, M. V. M. **Linguagens Formais e Autômatos.** Curso de Engenharia de Computação. Universidade Federal do Vale do São Francisco. 22 de abril de 2008. Disponível em: <http://docs.fct.unesp.br/docentes/dmec/olivete/lfa/arquivos/Apostila.pdf>. Acesso em: 27 dez. 2018.

SIPSER, M. **Introdução à Teoria da Computação**. 2.ed. São Paulo: Thompson Learning, 2007.

VIEIRA, Newton José. **Introdução aos fundamentos da computação:** Linguagens e máquinas. São Paulo: Thomson, 2006. 319 p.


Cruzeiro do Sul
Educacional